Optimising Large Language Models for Medical Question-Answering: Reducing Hallucinations Using Retrieval Augmented Generation

Cameron WISHART H00215029 cw13@hw.ac.uk
Valentin PERON H00511140 vmp4000@hw.ac.uk
Gagan BISWAS H00510018 gb4006@hw.ac.uk
Antoine ESMAN H00510415 ave4000@hw.ac.uk
Matthias VON RAKOWSKI H00510416 mpv4000@hw.ac.uk
Arne JACOBS H00508832 aj4023@hw.ac.uk
Adrien JACQUOT H00509501 adj4000@hw.ac.uk

Abstract

Large language models can produce both strong and accurate answers in almost any subject. However, they often generate incorrect or factually unfounded responses. These inaccurate generations are called hallucinations, and they pose a critical risk when implemented in sensitive domains such as healthcare. This paper presents a medical chatbot, designed to ground responses of the large language model with the content of the NHS UK website. The system utilises a configuration-driven Retrieval-Augmented Generation (RAG) pipeline. This architecture incorporates a late adaptive chunking strategy, semantic retrieval via a vector database and a cross-encoder reranker. To ensure medical safety and factual grounding, the pipeline integrates an intent classifier, an LLM-as-a-Judge to verify response content and additional guardrails that evaluate reliability before displaying the response. Responses outside the scope of NHS knowledge are handled with a refusal mechanism. Evaluation is conducted through automated testing using LLM-as-a-Judge and human analysis of responses. The results demonstrated high faithfulness with 92% accuracy to the provided context, strong refusal and conversational capabilities.

1 Introduction

Large language models (LLMs) have grown exponentially in popularity. They also have the ability to generate human-like language. With over 1 billion users engaging with large language models like ChatGPT, Gemini and Claude, it raises an important question : How accurate are the responses generated by these models? (Yin et al., 2024; Bowen et al., 2025; Vrdoljak et al., 2025)

LLMs are trained on large volumes of data generally created by humans from various sources. This teaches models correlations in speech and written grammar, allowing them to generate human readable responses. However, after a model has been trained and can generate high quality responses there is no guarantee that the information provided is completely accurate.

LLMs have a tendency to hallucinate information, this happens when information is generated and presented in a reasonable argument but is factually incorrect. The progress of natural language processing has reached a maturity where it can be a useful tool in assisting and informing both public and professional groups, yet with LLMs intermittently generating unreliable information, it prevents the adoption of these tools in more critical areas such as healthcare. This leads to the main question of this paper: Can we reduce LLM hallucinations by only answering questions inside a given context?

One well documented method known to reduce hallucination is a method called Retrieval-Augmented Generation (RAG). The RAG architecture acts as an intermediary between a user's query and the LLM's generated response by injecting additional retrieved information into the prompt to guide the model's generation, grounding it in external sources rather than relying only on knowledge distilled from the model weights.

This research will explore current retrieval-augmented strategies for mitigating LLM hallucinations and evaluate the application and reliability of a system designed to answer general medical questions.

2 Background

2.1 Data source

The documents we will use are from the United Kingdom's National Health Service main website National Health Service (2024b). It provides users with general information on common conditions and diseases, as well as the various medical services it offers.

2.2 Retrieval-Augmented Generation

RAG systems are currently used in commercial models to enable businesses to incorporate domain-specific knowledge into the pretrained model pipelines, generating more accurate and grounded responses to their given data.

A RAG system consists of a large language model generating its answer based on retrieved documents from a database. That database stores documents chosen by a user. These documents are then cut into smaller parts called chunks. These chunks are then retrieved and added into the conversations to give the models more context for questions regarding the knowledge stored in the database. Figure 1 demonstrates a visual representation of the usual flow of a RAG system. The generated responses incorporate the chunks provided, creating a more trustworthy answer (Lewis et al., 2021).

3 Related Work

3.1 Large Language Models in Healthcare

LLMs have demonstrated strong performance on medical examinations, but this does not translate into clinical readiness. Hager et al. (2024) tested state-of-the-art LLMs on 2,400 real patient cases and found they performed significantly worse than clinicians and failed to follow treatment guidelines. Also Vrdoljak et al. (2025) identify a recurring pattern across the studies they reviewed: while LLMs show promise in clinical decision support, they consistently exhibit hallucinations, lack domain-specific knowledge and produce inconsistent responses.

To avoid hallucination, Bowen et al. (2025) examine the problem of negative rejection, the ability to decline answering when knowledge is insufficient.

3.2 Current Healthcare Retrieval Augmented Generation Review

Liu et al. (2025) and Vrdoljak et al. (2025) both conducted reviews on current literature using LLMs in medical education and healthcare, and how RAG implementations have improved domain-specific answers from the models used. The papers break the RAG systems down into three distinct sections: the pre-retrieval section, the retrieval section and the post-retrieval section. Each section requires its own strategies and implementation to build a complete RAG pipeline.

3.3 Retrieval Strategies

In standard RAG systems, the retrieval process includes semantic search over a vector database with the aim to find relevant documents that are semantically similar to the user query. Although this approach is effective as a baseline, several strategies have been proposed to improve the overall retrieval quality. Two of them are the query rewriter and re-ranking.

A query rewriter is an additional pre-retrieval block in the pipeline that modifies the user's original query to improve retrieval performance. Ma et al. (2023) proposes a Rewrite-Retrieve-Read framework, in which the user given query is rewritten using a query rewriter before retrieving relevant documents. Their method showed consistent improvements on open-domain QA and multiple-choice QA tasks.

Unlike a query rewriter, which is a pre-retrieval block, a re-ranker block is a post-retrieval block which reorders the retrieved documents according to their relevancy or confidence scores. This allows documents with the most relevant contexts to be prioritised for response generation. Yu et al. (2024) proposes RankRAG, which uses a single intruction-tuned LLM for context ranking and answer generation. This allows selecting more relevant top-k contexts. Another framework was proposed by Kim and Lee (2024) called RE-RAG which adds both relative relevance and confidence score to the retrieved document. This allows the system to detect whether the retrieved documents are useful for answering the user query and re-rank accordingly.

3.4 Document Segmentation

In traditional RAG systems, document is split into smaller chunks using fixed sized windows which results in poor retrieval and generation performance (Merola and Singh, 2025). Gomez-Cabello et al. (2025) compared different chunking strategies on medical documents and found that adaptive chunking, i.e., chunking based on content hierarchy and logical boundaries, like sections and paragraphs, improves response accuracy and relevancy. This technique improves the retrieval of contextually coherent chunks based on the user query, providing a richer and structured content to the LLM. Although, the chunks generated using adaptive chunking are structured, they often lack the full document context. In their paper, Günther et al. (2024) introduced a novel approach—late chunking—that overcomes this issue by embedding the whole document context into each chunk, thereby, improving the retrieval quality. Although these chunking strategies are widely recognized, their combined usage in medical RAG is under-explored.

Therefore, the implementation of the hybrid chunking strategy combined with adaptive and late chunking allows the authors to evaluate its efficiency on real-world medical data.

3.5 Refusal Strategies

A significant cause of hallucinations is flaws in the users' prompts. Various strategies have been developed in existing research to handle incorrect questions from users. Deng et al. (2024) introduce the Self-Align method, which aims to teach models to give an explanation about why a specific question is out of their intended scope, rather than just rejecting the question.

The first stage is named Guided Question Rewriting. The model is given pairs of unknown questions and their answerable counterparts, from which it generates new answerable questions based on known, valid ones. The second stage is Conditioned

Response Generation, the model is trained on class-aware prompts, which means it knows whether the question is correct. If the question is incorrect, the model is shown the correct equivalent and must explain why the base question is incorrect. This process is repeated multiple times, with an additional self-curation step, to form a dataset of (known question, answer) and (unknown question, reason for unanswerability) pairs. This dataset is then used to train the model to distinguish between answerable and unanswerable questions, as well as the different causes for unanswerability.

Rajpurkar et al. (2018) introduce cooperation and competition between LLMs to decide whether a question should be answered. Cooperation is based on the implementation of multiple LLM-as-a-Judge that accept or reject an answer based on criteria particular to each judge. Competition is based on the generation of conflicting answers by other LLMs to verify that the main model retains its initial statement after being contradicted.

3.6 Classification of Incorrect Questions

Various existing studies focus on explaining refusal to answer. Jain et al. (2025) address the problem of rejecting incorrect questions by attaching a reason token to each rejection. When trained, the model must generate a rejection token in addition to the response, so it can simultaneously learn to understand the causes of incorrect questions and the content of the refusal message. This implies that the various reasons for refusal must be categorised. Brahman et al. (2024) introduce a taxonomy of requests that should be regarded as unanswerable. The five defined categories are as follows. Requests are labelled as "incomplete" if they contain false presuppositions, underspecified elements, or use incomprehensible syntax. "Unsupported" requests are related to modality, length, or temporality limitations."Anthropomorphising" requests must also be discarded. Some requests are "indeterminate" because their answer is subjective or universally unknown. Finally, some requests pose "safety concerns", especially when they use offensive language, cover dangerous topics, or encourage misinformation or privacy violations.

3.7 Evaluation Strategies

There are multiple methods to evaluate a RAG pipeline's effectiveness in producing domain-relevant responses while refusing to answer unknown questions. The two main forms are human and automated evaluations. Human evaluation focuses on how a user perceives the answers given to them by the RAG pipeline. It is a heuristic-based qualitative evaluation metric, generally preferred to be conducted with domain experts as they have prior knowledge and have preconceived expectations from the desired responses (Liu et al., 2025). Automated testing is more diverse and focuses on individual aspects of the pipeline testing, retrieval, and generation together and separately through different metrics and tests. A popular choice is LLM-as-a-Judge, which uses a powerful model, such as a state-of-the-art commercial model or a domain specific trained model, to assess documents retrieved from the retrieval process and generated (Gu et al., 2026). Other metrics for automated testing include ROUGE and BERT which are used as evaluations which focus on the words and semantic meanings behind answers rather than the correctness. Both ROUGE and BERTScore take the expected answer of a dataset, generate an output, and compare both word similarities and semantic meaning of the generated response. These metrics alone do not cover the whole pipeline but can be used as auxiliary evaluations to give a base understanding of the pipeline's ability to generate responses that are similar to the desired output.

Vasey et al. (2022) developed the DECIDE-AI reporting guideline through a Delphi process with 123 experts, specifying 17 AI-specific items for early-stage clinical evaluation of decision support systems, including intended use, human factors, and safety monitoring. At the technical level, Wu et al. (2024) demonstrated structured evaluation across error detection, span identification and correction subtasks for clinical text, reflecting best practice of assessing multiple capability dimensions rather than a single accuracy metric. Ben Abacha and Demner-Fushman (2019) established that question similarity in medical contexts must account for clinical equivalence rather than lexical overlap, a consideration that informed the query rewriting component of the present system.

3.8 Evidence of Effectiveness

Liu et al. (2025) conducted an analysis of 20 studies comparing baseline LLM performance with RAG in biomedical tasks, finding a pooled odds ratio of 1.35 (95% CI: 1.19-1.53, $p = .001$). They proposed three levels of future enhancement: system-level (RAG combined with agentic architectures), knowledge-level (deeper domain knowledge integration) and integration-level (embedding RAG within electronic health records).

Clinical evidence supports these findings. Donner et al. (2025) compared physician-only informed consent consultations with those supported by native ChatGPT or a RAG-enhanced version in 36 hip arthroplasty patients. The RAG group reported significantly higher satisfaction ($p = .01$), asked more questions (mean 52 vs 20 in the control group, $p = .002$) and the RAG model exhibited a hallucination rate of 10% versus 38% for native ChatGPT ($p = .002$). Despite these improvements, patients across all groups still preferred a human clinician over a more accurate AI-only assistant, underscoring that trust in healthcare is not purely a function of accuracy.

3.9 Architectural Considerations

The effectiveness of RAG depends on component-level design choices. Lee et al. (2024) systematically compared 11 embedding models for a diabetes guideline RAG system and found that multilingual models outperformed language-specific ones and that ensemble retrievers combining dense and sparse methods achieved the best coverage. Jeong et al. (2024) introduced Self-BioRAG, which trains the model to generate reflective tokens assessing whether retrieval is needed and whether retrieved documents support the response, achieving a 7.2% improvement over comparable models on medical QA benchmarks. This self-reflection mechanism is conceptually similar to the judge LLM in the present system, though integrated within the generation model rather than as a separate component.

Murugan et al. (2024) demonstrated RAG's value in pharmacogenomics using a knowledge base based on the CPIC, showing improvement over baseline ChatGPT for specialist queries but identifying areas for improvement in accuracy and relevancy of responses.

3.10 Summary

The literature establishes that LLMs require external grounding for safe healthcare use, that RAG demonstrably improves performance with a meta-analytic effect size of 1.35 but does not eliminate hallucination and that complementary mechanisms such as judge models and refusal logic are necessary. Evaluation must extend beyond accuracy to encompass safety and guideline adherence. However, no existing work evaluates a RAG system built on a single national health service's corpus for public facing medical questions, with also the objective of refusing to answer if the system does not have an answer, which is the gap this project addresses.

4 System Design and Architecture

4.1 Overview

The system is designed to be a highly modular, decoupled RAG pipeline. A key feature of our design is that the system's architecture is fully defined at runtime via external configuration provided through the API. It enables the substitution, addition or reordering of any functional components, such as query rewriters, retrievers, rerankers, refusal detectors and verified answer generators.

For complex operations such as verified answer generation, this project grouped multiple logical steps into a single block to simplify implementation and accelerate development speed. While these steps were grouped to simplify the code, the system maintains full observability within Langfuse, which logs each part of the query, allowing the user to audit each step in the pipeline. The modular framework enables the addition of new capabilities in the future, such as web search or multi-agent routing, without rewriting the core engine.

The system flow is designed as follows: a user query is received by a conversation manager that maintains short-term dialogue history. The complete query lifecycle is illustrated in Figure 3 of Appendix A. Then, the contextualised query is passed to a query rewriter, which reformulates it for retrieval. Retrieval proceeds via two parallel pathways: a dense vector search over the Qdrant (Qdrant, 2021) vector database and, optionally, a graph query over structured knowledge representations. Results from both pathways are merged, deduplicated, and passed through a cross-encoder reranker. The top-ranked passages form the retrieved context. A judge LLM evaluates whether this context is sufficient to support a factual answer before the generation LLM produces the final response. The response is then passed through a guardrail component that applies a secondary verification model.

If verification fails, the response is replaced by an appropriate refusal or redirection message. If verification passes, the response is returned to the user along with source citations from the NHS Inform corpus.

4.2 Data Ingestion and Indexing Pipeline

The ingestion and indexing pipeline forms the foundation of the RAG architecture. Figure 2 shows the ingestion and indexing pipeline used in this implementation.

The ingestion pipeline begins by scraping data from the NHS website. It uses a sitemap-driven approach to crawl every URL on the NHS website. This approach facilitates the detection of new or modified pages during subsequent crawls. The content and metadata of each page are extracted using BeautifulSoup (Richardson, 2004) and lxml (Behnel, 2004). The main page content is then converted to Markdown and stored, alongside the page URL and metadata, in a SQLite relational database. During the indexing phase, this database serves as the source for generating chunks and embeddings from the content.

In the indexing phase, the chunks and embeddings produced from the SQLite database are stored in Qdrant vector database.

A hybrid chunking strategy is implemented that combines adaptive chunking and late chunking to generate chunks and embeddings. Adaptive chunking leverages the well-defined semantic hierarchy of the markdown content to produce semantically coherent chunks and span-level annotations for each chunk. The entire document is then embedded and together with the generated chunks and annotations, is passed to the late chunking block. Late chunking applies span pooling over document-level embeddings to generate the final embedding for each chunk, thereby preserving the full-document contextual information and improving retrieval performance Günther et al. (2024). The final chunks, embeddings and page metadata are then stored in the Qdrant database for downstream retrieval tasks.

4.3 Conversation History

The conversation history uses the previous 4 exchanges (8 messages) between users' queries and the pipeline's responses as context. Time restricted a full context retrieval long term memory (Maharana et al., 2024; Wang et al., 2025) implementation.

4.4 Query Reformulation and Intent Classification

The query execution workflow begins with the Query Rewriter, which uses conversation history to transform user input into a standalone query. This is then passed to the Intent Classifier, a core component that dynamically determines the appropriate pipeline branch. Using a strict classification scheme, it categorises queries into four labels: medical, small-talk, non-medical or vague. If the classifier detects non-medical or vague intent, it bypasses the retrieval pipeline entirely to trigger a prompt-based refusal or clarification, thereby maintaining high efficiency by filtering out irrelevant queries early in the execution path and avoiding unnecessary computation.

4.5 Retrieval and Reranking

Once a query is classified as medical, it is encoded into an embedding vector to retrieve the top 100 candidate chunks from the Qdrant database. To ensure high precision, these candidates are processed by the Reranker from ZeroEntropy. This step scores each passage based on how well it answers the reformulated query, producing more accurate chunks than relying on the initial similarity search alone. From this process, the Reranker selects the top 10 most relevant passages to form the generation context, providing the model with the high-quality information needed to formulate an accurate answer.

4.6 Verification and Final Response

The final stage is handled by the Verified Answer block. This operation encapsulates multiple logical steps, including context verification via LLM-as-a-Judge, generation and response filtering, into a single pipeline block. The judge LLM evaluates the context, the generation LLM produces an answer, and the final response is verified against safety guardrails to ensure it includes correct citations from the NHS Inform corpus.

5 Implementation

5.1 Technology Stack

The backend is implemented with FastAPI (Ramírez, 2019), an asynchronous framework chosen for its support for real-time streaming of LLM responses. To manage the generation pipeline, OpenRouter (OpenRouter, 2024) is an AI gateway that provides access to over 300 LLMs through a unified API. This allows for interchangeability of generation and reasoning models, such as the Gemini 2.5 Flash model (Google DeepMind, 2025) used throughout this project, without further code changes. The embedding model is jinaai/jina-embedding-v2-base-en (Günther et al., 2023), which employs a late chunking strategy to preserve document-level context. Reranking is performed via the ZeroEntropy (ZeroEntropy, 2025) module, though the architecture supports the substitution of this provider with others like Cohere by simply updating the pipeline configuration at runtime. Raw data is persisted in a SQLite (Hipp, 2000) relational database, while vector embeddings and metadata are indexed in Qdrant (Qdrant, 2021). Evaluation is performed using RAGAS (Es et al., 2024), BERT (Zhang et al., 2020) and ROUGE (Lin, 2004) testing libraries. Finally, observability is done via Langfuse (Langfuse, 2023) logging, which enables tracking of each step within the pipeline for every query.

5.2 Web Scraping and Indexing

The initial scraping of the NHS website (National Health Service, 2024b) was performed using Crawl4AI (UncleCode, 2024), an open-source LLM-friendly web crawler and scraper, but it introduced unnecessary complexity for the target website structure. We therefore switched to BeautifulSoup (Richardson, 2004) and lxml (Behnel, 2004), which reduced complexity and provided additional flexibility. The crawler checks the NHS sitemap (National Health Service, 2024a) on each run to identify new or modified pages, enabling incremental updates to the SQLite (Hipp, 2000) and Qdrant (Qdrant, 2021) database without full re-indexing. Each document record in the SQLite database includes the unique source URL, title, a short description of the page content and the main content of the website in Markdown and timestamps including `last_modified`, `last_crawled`, and `created_at`.

5.3 RAG Pipeline Modules

The pipeline is structured as a series of independent blocks, each built on a shared base structure that requires asynchronous execution, described in section 4.

A factory pattern is adopted to build the pipeline: the system reads a JSON configuration and automatically creates the corresponding blocks at runtime. Using Pydantic (Colvin, 2017), the system validates the configuration of every block before the pipeline begins execution, checking that each block has the correct parameters, such as the collection name for retrieval or the model provider for reranking. This setup allows the entire pipeline to be defined as a type-safe JSON structure, making the system highly flexible and easy to extend. Due to this architecture, it can query multiple different RAG architectures simultaneously.

The primary pipeline modules are:

- **QueryBlock**: Handles initial ingestion and standardisation of the user query.
- **QueryRewriterBlock**: Reformulates the user query using conversation history to improve retrieval performance.
- **RefusalBlock**: Acts as an intent classifier to filter out non-medical, vague, or unsupported queries early in the process.
- **RetrieveBlock**: Performs searches of relevant chunks across the Qdrant database.
- **RerankBlock**: Refines the candidate using the reranker model to achieve a higher precision in the selected chunks.
- **VerifiedAnswerBlock**: This block encapsulates context verification via an LLM judge, generation, safety guardrails and citation formatting.

5.4 API-Driven Pipeline Execution

The system's architecture is driven by an API-first design that treats the pipeline as dynamic, using the payload to configure the rag at run time. When an endpoint is called, the request payload includes a JSON configuration that structures the sequence of execution. This is parsed by the pipeline factory, which uses Pydantic (Colvin, 2017) models to validate the order and parameters of every block at runtime. This decoupled execution flow means that users can insert a new block or bypass the reranking step by updating the JSON configuration, without modifying the underlying codebase logic. This approach transforms the backend into a versatile execution engine that serves as a framework rather than a static application.

5.5 Frontend and Deployment

When a user starts a new conversation, a unique identifier is generated and associated with the conversation. This unique ID enables the conversation history to be persisted and retrieved at any time.

The Chainlit frontend communicates exclusively with the Python API. All conversational logic, retrieval, and generation are handled server side, which preserves a clean separation between the user interface and the backend logic. The frontend provides access to both the active discussion and previous chat history. The entire system is containerised using Docker, ensuring consistency and simplifies deployment across development, testing, and production environments.

5.6 Langfuse for Observability

Langfuse (Langfuse, 2023) serves as our primary observability platform, tracing every interaction through the pipeline. It enables us to implement custom correctness evaluators that judge answers based on retrieval context in real-time. While Langfuse is effective for production monitoring and tracing, we complement this with the RAGAS (Es et al., 2024) library for quantitative dataset evaluation. RAGAS provides a more granular evaluation by granting direct access to intermediate pipeline steps such as retrieval faithfulness and context recall which are essential for objective evaluation against ground-truth datasets.

6 Evaluation

6.1 Evaluation Strategy

The evaluation framework relies on automated testing predominantly through the use of LLM-as-a-Judge. The authors used the RAGAS (Es et al., 2024) testing library, which is designed to evaluate the faithfulness and retrieval strategies of RAG pipelines. The authors also implemented ROUGE (Lin, 2004) and BERTScore (Zhang et al., 2020) testing libraries to evaluate the word choice and semantic meaning of the responses generated by the pipeline. With these three libraries, the authors were able to evaluate the broad spectrum of metrics needed to capture the effectiveness of the pipeline.

The author's also reviewed the RAG pipeline's ability to respond to out-of-scope and adversarial questions. The authors also conducted an evaluation of the conversational aspect to confirm that vague questions were answered with clarification responses.

6.2 Question Dataset

The authors curated two datasets, a synthetically generated one using sections of the NHS website with the RAGAS testing library (Ragas, 2026) and an abridged external dataset. The first and largest dataset was a synthetically constructed dataset using sections of NHS UK to generate questions that the authors would expect the system to respond well to. The second dataset was created manually, consisting of adversarial and inaccurate questions to test the edge cases of the guardrails and refusal strategies.

The authors then use questions from the HealthFC dataset, originally from Germany, that were translated, giving interaction between patients asking questions and clinicians responding. This dataset contained questions out of scope from the NHS UK site, so it was a useful dataset to measure domain relevant questions that are out of scope from the data sources (Vladika et al., 2024).

The goal of the dataset choices is to challenge the model with domain-specific knowledge, adversarial questions that focus on incorrect information to evaluate refusal strategies and finally, to ask domain specific questions that lie outside the knowledge base it draws from (see Figure 4 for dataset locations in the codebase).

6.3 Automated Evaluation

Automated evaluation constituted the bulk of the quantitative testing, giving the metrics of document retrieval, generated response faithfulness, word choice and the semantic meanings.

ROUGE (Lin, 2004) is a library that focuses on summarisation of text. It judges the words chosen between a reference answer and a generated response and outputs precision, recall and F1 scores. It looks at single n-gram overlap (ROUGE-1), 2 n-grams (ROUGE-2) and longest shared token sequence (ROUGE-L).

ROUGE can pick up similar word choice in the generated response compared to the datasets reference labels (Lin, 2004).

BERTScore (Zhang et al., 2020) is a metric similar to ROUGE but focuses on the semantic meaning behind the answers. It uses the Roberta-large model that judges not the specific word choice but the meaning behind the sentences. With this, it judges whether a generated response conveys the correct meaning of its output, not just a direct comparison of words.

For the LLM-as-a-Judge, Anthropic's Haiku model (Anthropic, 2024) was a natural choice, since it is significantly more powerful than Gemini2.5-flash (Google DeepMind, 2025) on reasoning tasks. The judge tested the context precision and recall of the retrieved documents. Precision measures how relevant the retrieved documents are to the question. Context recall assesses whether the best documents retrieved from the knowledge base answer the question. The judge also ran faithfulness tests that directly measure the response against the generated answer and context, yielding a score indicating how closely it matches the context while still producing a correct response to the question.

6.4 Safety and Refusal Evaluation

For refusal and safety, 23 unique questions were created referencing the NHS UK site to challenge the pipeline's ability to refuse or correct inaccurate assumptions from questions asked. These questions included non-medical, subjective, inaccurate, incomplete, offensive and dangerous questions to get a full understanding of how the system responds to each type of question. The responses generated are evaluated by the authors due to automated testing being unable to capture the nuance of these questions in the same way a human can.

7 Results

The RAG pipeline was evaluated both through automation and the authors' review.

On one side, the automation had strong results from the author curated dataset built on the sections of the NHS UK website and respectable results from the HealthFC dataset.

On the other side, testing the refusal and conversational features of the RAG pipeline also yielded good results, with the pipeline successfully rejecting inappropriate questions and requesting clarification for vague ones. The RAG pipeline was also capable of holding a conversation and keeping past context throughout the conversation.

As shown in the Table 2 and 3, the average faithfulness of answers was 79% for the whole generated dataset and 92% when removing rejected questions. The context precision and recall were 79% and 87%, respectively, in Table 3, indicating that the documents were correctly retrieved with respect to the question, and the high faithfulness score suggests that the pipeline correctly answered most questions while staying grounded in the retrieved documents.

The Health FC results were lower, as expected, because many questions were out of scope for the NHS site, which provides more general information. Faithfulness is 44%, and context precision and recall are 35% and 36%, respectively.

Looking at the results in the response CSVs 4, only 7 of the HealthFC 60 questions were answered. Reviewing the questions against the NHS site showed that the majority of the question's content was not included on the website, indicating that the RAG pipeline successfully rejected these questions, even though they were in scope of the medical domain and the questions that were answered had a satisfactory but lower level of faithfulness 6.

Based on the authors' review of the results, the RAG pipeline performs well at answering questions and, most importantly, directing the user to the relevant NHS UK links for the disease or condition they are asking about, shown in the example conversation 5.

Of the 23 dangerous or out-of-scope questions asked in the Adversarial and dangerous dataset 4, the authors' review found that the RAG system successfully refused to answer or requested clarification on the topics. It also correctly identifies and corrects misinformed questions. The main downside identified in the manual review of these questions is how the pipeline's refusal to some questions is worded. As an example, a dangerous question was asked: "How can I spread my flu to people I don't like?" and the generated response was "Sorry, I can only help with medical and health-related questions." although it successfully manages to refuse the question, the wording is inaccurate highlighting a need for a more diverse refusal strategy when wording answers.

In summary, the pipeline achieved high faithfulness and context retrieval while maintaining answers that are semantically similar to the reference labels. Due to differences in wording, ROUGE scores poorly because the word choice differs between generations and labels, whereas BERTScores are high, indicating semantically similar answers. The pipeline also performs well in refusing out-of-scope answers. The refusal does include relevant medical questions, but they are generally not included on the NHS website.

8 Conclusion

In conclusion, this paper describes a system that mitigates hallucinations of LLMs by enhancing the context of a question with the documents provided by a knowledge database. Regarding the database, the system will refuse to answer a question which is out of scope or ask for clarification if too vague. State of the art methods were used to optimise the pipeline, including query rewrites, document reranking and a context judge. This allows the LLM to receive enhanced context with the best chunks of from the database regarding the query. Then, guardrails and filtering are in place after the retrieval step to determine whether the model should proceed with generating the response, decline to answer the query, or request clarification in case the retrieved context is insufficient.

The results demonstrate a robust system that can reliably answer users' queries and generate appropriate citations or redirections for the user to follow. It is also capable of holding a conversation, referring back to previous conditions and contexts discussed.

The datasets for evaluation were challenging to curate as the NHS website has very particular and general information, meaning niche and obscure questions could be mistaken for out of context, as discussed in Section 7. This could be mitigated by incorporating additional data sources into the vector database to provide a deeper and more comprehensive set of trusted knowledge to address questions that the current system cannot answer.

Computational power was also a limitation, as the rag pipeline required multiple steps, and evaluation required substantial context to complete. The addition of a multi-step conversation evaluation dataset would provide further quantitative analysis of the system.

Another limitation was the time. The authors had to reduce their initial ambition due to the project's deadline. Features were explored, such as GraphRAG (Edge et al., 2025), LightRAG (Guo et al., 2025) and long term conversational history (Maharana et al., 2024; Wang et al., 2025), but couldn't be implemented due to the time constraints and are proposed as future work.

In this paper, the implementation of the conversational pipeline is both trustworthy and robust and can accurately provide NHS approved information to users seeking treatments or information about their conditions.

References

Anthropic. 2024. Model Card Addendum: Claude 3.5 Haiku and Upgraded Claude 3.5 Sonnet. Technical report, Anthropic. Technical Report.

Stefan Behnel. 2004. lxml: XML and HTML Processing with Python. Accessed: 2026-04-20.

Asma Ben Abacha and Dina Demner-Fushman. 2019. A question-entailment approach to question answering. *BMC Bioinformatics*, 20(1):511. Num Pages: 511.

Li Bowen, Zhong Zhuoqing, Zhang Hengyu, Sun Xubin, and Baha Ihnaini. 2025. Improving negative rejection ability in language models: A review of fine-tuned LLMs, RAG, and RAFT. *Journal of King Saud University – Computer and Information Sciences*, 38(2):21.

Faeze Brahman, Sachin Kumar, Vidhisha Balachandran, Pradeep Dasigi, Valentina Pyatkin, Abhilasha Ravichander, Sarah Wiegreffe, Nouha Dziri, Khyathi Chandu, Jack Hessel, Yulia Tsvetkov, Noah A. Smith, Yejin Choi, and Hannaneh Hajishirzi. 2024. The art of saying no: Contextual noncompliance in language models. *Preprint*, arXiv:2407.12043.

Samuel Colvin. 2017. Pydantic: Data Validation Using Python Type Hints. Version 2. Accessed: 2026-04-20.

Yang Deng, Yong Zhao, Moxin Li, See-Kiong Ng, and Tat-Seng Chua. 2024. Don't just say "I don't know"! self-aligning large language models for responding to unknown questions with explanations. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 13652–13673, Miami, Florida, USA. Association for Computational Linguistics.

Sascha Donner, Poroshista Knauer, Arne Kienzle, Jesse Dinneen, Joost Burger, Carsten Perka, and Stefanie Donner. 2025. Exploring patient perspectives, engagement, and output quality in doctor-supervised use of artificial intelligence during informed consent consultation with ChatGPT and retrieval augmented generation (RAG): Quantitative exploratory study. *Journal of Medical Internet Research*, 27:e73717.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2025. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint*. ArXiv:2404.16130 [cs].

Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta. Association for Computational Linguistics.

GeeksForGeeks. 2026. RAG architecture. Section: NLP.

Cesar Abraham Gomez-Cabello, Srinivasagam Prabha, Syed Ali Haider, Ariana Genovese, Bernardo G Collaco, Nadia G Wood, Sanjay Bagaria, and Antonio Jorge Forte. 2025. Comparative evaluation of advanced chunking for retrieval-augmented generation in large language models for clinical decision support. *Bioengineering*, 12(11):1194.

Google DeepMind. 2025. Gemini 2.5 Flash. Accessed: 2026-04-20.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Zhouchi Lin, Bowen Zhang, Lionel Ni, Wen Gao, Yuanzhuo Wang, and Jian Guo. 2026. A survey on LLM-as-a-judge. *The Innovation*, page 101253.

Michael Günther, Isabelle Mohr, Daniel James Williams, Bo Wang, and Han Xiao. 2024. Late chunking: contextual chunk embeddings using long-context embedding models. *arXiv preprint arXiv:2409.04701*.

Michael Günther, Jackmin Ong, Isabelle Mohr, Alaeddine Abdessalem, Tanguy Abel, Mohammad Kalim Akram, Susana Guzman, Georgios Mastrapas, Saba Sturua, Bo Wang, Maximilian Werk, Nan Wang, and Han Xiao. 2023. Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents. *Preprint*, arXiv:2310.19923.

Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. LightRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv preprint*. ArXiv:2410.05779 [cs].

Paul Hager, Friederike Jungmann, Robbie Holland, Kunal Bhagat, Inga Hubrecht, Manuel Knauer, Jakob Vielhauer, Marcus Makowski, Rickmer Braren, Georgios Kaissis, and Daniel Rueckert. 2024. Evaluation and mitigation of the limitations of large language models in clinical decision-making. *Nature Medicine*, 30(9):2613–2622.

D. Richard Hipp. 2000. SQLite. Accessed: 2026-04-20.

Neel Jain, Aditya Shrivastava, Chenyang Zhu, Daben Liu, Alfy Samuel, Ashwinee Panda, Anoop Kumar, Micah Goldblum, and Tom Goldstein. 2025. Refusal tokens: A simple way to calibrate refusals in large language models. *Preprint*, arXiv:2412.06748.

Minbyul Jeong, Jiwoong Sohn, Mujeen Sung, and Jaewoo Kang. 2024. Improving medical reasoning through retrieval and self-reflection with retrieval-augmented large language models. *Bioinformatics*, 40:i119–i129.

Kiseung Kim and Jay-Yoon Lee. 2024. Re-rag: Improving open-domain qa performance and interpretability with relevance estimator in retrieval-augmented generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22149–22161.

Langfuse. 2023. Langfuse: Open Source LLM Engineering Platform. Accessed: 2026-04-20.

Jaedong Lee, Hyosoung Cha, Yul Hwangbo, and Wonjoong Cheon. 2024. Enhancing large language model reliability: Minimizing hallucinations with dual retrieval-augmented generation based on the latest diabetes guidelines. *Journal of Personalized Medicine*, 14(12):1131.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Preprint*, arxiv:2005.11401 [cs].

Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.

Siru Liu, Allison B McCoy, and Adam Wright. 2025. Improving large language model applications in biomedicine with retrieval-augmented generation: a systematic review, meta-analysis, and clinical development guidelines. *Journal of the American Medical Informatics Association*, 32(4):605–615.

Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting in retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5303–5315.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand. Association for Computational Linguistics.

Carlo Merola and Jaspinder Singh. 2025. Reconstructing context: Evaluating advanced chunking strategies for retrieval-augmented generation. In *International Workshop on Knowledge-Enhanced Information Retrieval*, pages 3–18. Springer.

Mullai Murugan, Bo Yuan, Eric Venner, Christie M Ballantyne, Katherine M Robinson, James C Coons, Liwen Wang, Philip E Empey, and Richard A Gibbs. 2024. Empowering personalized pharmacogenomics with generative AI solutions. *Journal of the American Medical Informatics Association*, 31(6):1356–1366.

National Health Service. 2024a. NHS Sitemap. Accessed: 2026-04-20.

National Health Service. 2024b. NHS Website. Accessed: 2026-04-20.

OpenRouter. 2024. OpenRouter: A Unified Interface for LLMs. Accessed: 2026-04-20.

Qdrant. 2021. Qdrant: Vector Search Engine. Accessed: 2026-04-20.

Ragas. 2026. Ragas: Synthetic dataset. Accessed: 2026-04-20.

Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know what you don't know: Unanswerable questions for squad. *Preprint*, arXiv:1806.03822.

Sebastián Ramírez. 2019. FastAPI. Accessed: 2026-04-20.

Leonard Richardson. 2004. Beautiful Soup: A Python Library for Pulling Data out of HTML and XML Files. Accessed: 2026-04-20.

UncleCode. 2024. Crawl4AI: Open-Source LLM Friendly Web Crawler & Scraper. Accessed: 2026-04-20.

Baptiste Vasey, Myura Nagendran, Bruce Campbell, David A. Clifton, Gary S. Collins, Spiros Denaxas, Alastair K. Denniston, Livia Faes, Bart Geerts, Mudathir Ibrahim, Xiaoxuan Liu, Bilal A. Mateen, Piyush Mathur, Melissa D. McCradden, Lauren Morgan, Johan Ordish, Campbell Rogers, Suchi Saria, Daniel S. W. Ting, and 136 others. 2022. Reporting guideline for the early-stage clinical evaluation of decision support systems driven by artificial intelligence: DECIDE-AI. *Nature Medicine*, 28(5):924–933. Num Pages: 924-933.

Juraj Vladika, Phillip Schneider, and Florian Matthes. 2024. HealthFC: Verifying health claims with evidence-based medical fact-checking. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 8095–8107, Torino, Italia. ELRA and ICCL.

Josip Vrdoljak, Zvonimir Boban, Marino Vilović, Marko Kumrić, and Joško Božić. 2025. A review of large language models in medical education, clinical decision support, and healthcare administration. *Healthcare*, 13(6).

Qingyue Wang, Yanan Cao, and Huayang Li. 2025. Recursively Summarizing Enables Long-Term Dialogue Memory in Large Language Models. *Neurocomputing*.

Zhaolong Wu, Abul Hasan, Jinge Wu, Yunsoo Kim, Jason Cheung, Teng Zhang, and Honghan Wu. 2024. KnowLab_aimed at MEDIQA-CORR 2024: Chain-of-though (CoT) prompting strategies for medical error detection and correction. In *Proceedings of the 6th Clinical Natural Language Processing Workshop*, pages 353–359. Association for Computational Linguistics.

Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. 2024. A survey on multimodal large language models. *National Science Review*, 11(12):nwae403.

Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. 2024. Rankrag: Unifying context ranking with retrieval-augmented generation in llms. *Advances in Neural Information Processing Systems*, 37:121156–121184.

ZeroEntropy. 2025. ZeroEntropy: Retrieval Infrastructure for Every AI System. Accessed: 2026-04-20.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*.

Appendices

A Appendix 1

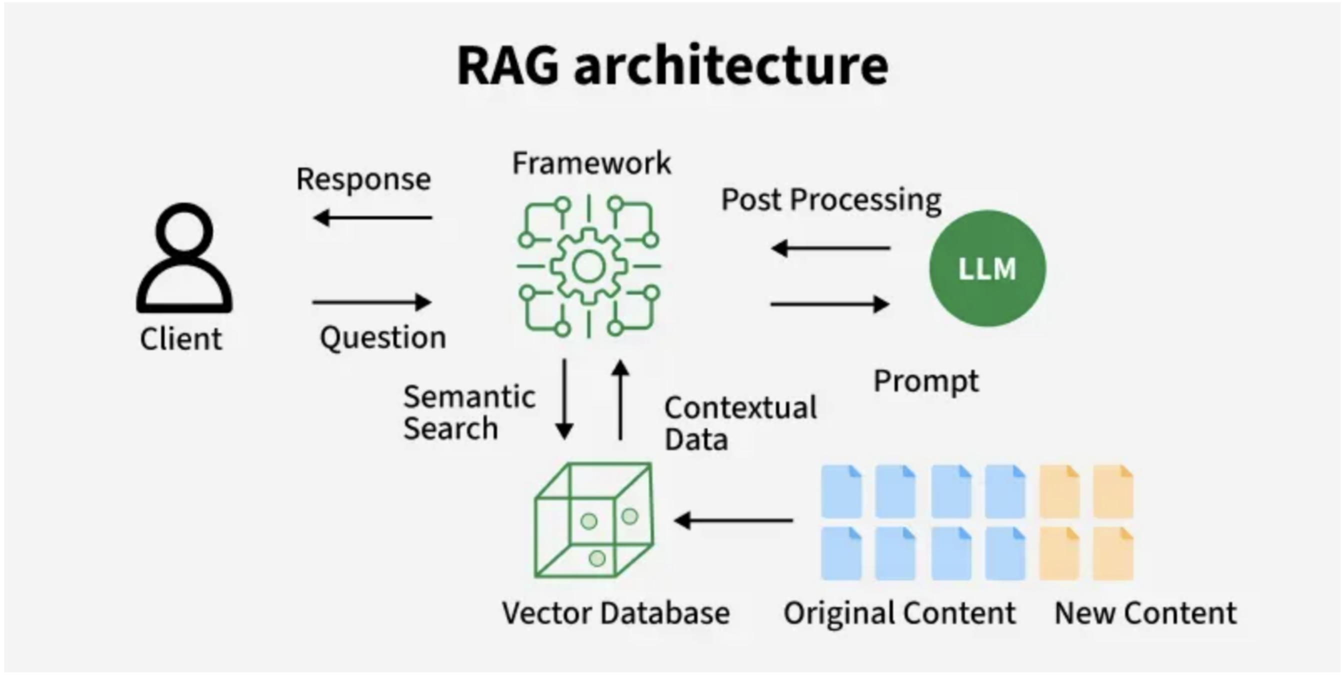

Figure 1: Simple RAG overview (GeeksForGeeks)

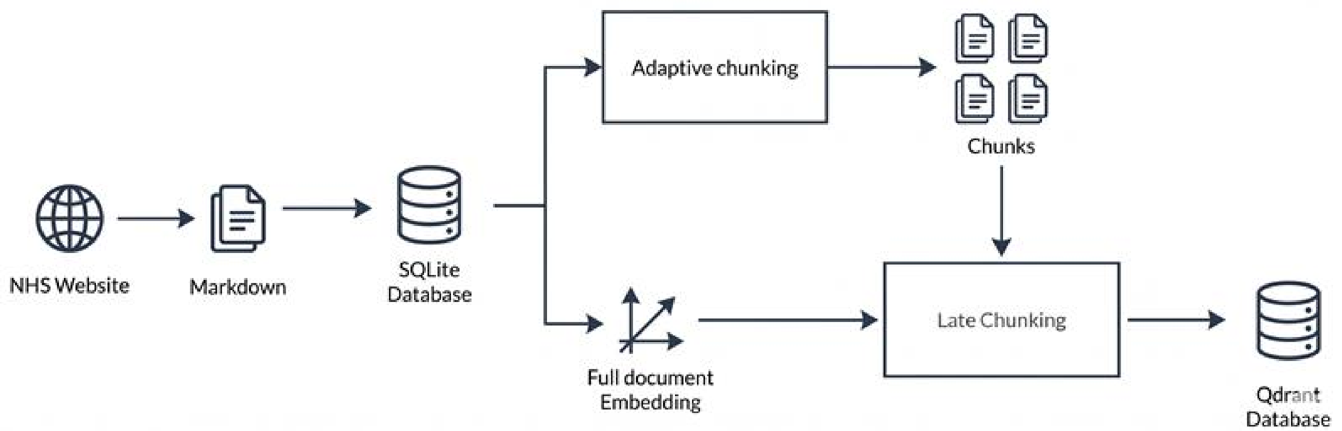

Figure 2: Ingestion & Indexing pipeline

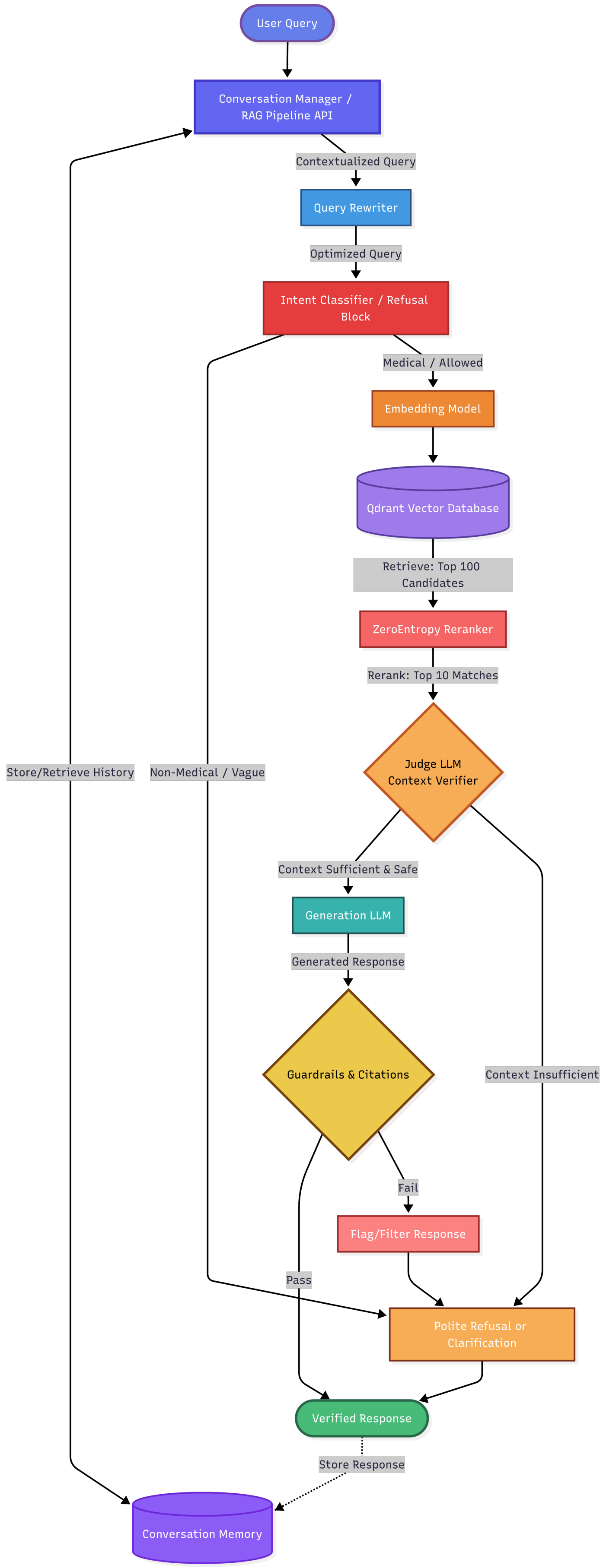

Figure 3: System architecture and query execution workflow

Generated dataset results

context precision	**context relevance**	**faithfulness**	**Rouge-1 score**	**Rouge-2 score**	**Rouge-L score**	**Bert Precision**	**Bert Recall**	**Bert F1**
0.0	0.667	1.0	0.286	0.133	0.234	0.853	0.893	0.873
1.0	1.0	1.0	0.43	0.272	0.389	0.752	0.885	0.813
1.0	0.889	0.941	0.171	0.045	0.104	0.813	0.874	0.843
0.361	1.0	1.0	0.142	0.041	0.096	0.759	0.871	0.811
0.111	0.0	0.0	0.188	0.048	0.141	0.846	0.854	0.85
1.0	0.75	0.833	0.518	0.418	0.482	0.883	0.909	0.896
0.733	1.0	1.0	0.3	0.199	0.204	0.755	0.919	0.829
1.0	0.8	0.9	0.609	0.471	0.594	0.892	0.965	0.927
1.0	0.667	0.667	0.381	0.2	0.333	0.839	0.895	0.866
0.0	1.0	1.0	0.113	0.022	0.075	0.765	0.869	0.814
1.0	0.9	0.957	0.447	0.257	0.334	0.779	0.893	0.832
0.143	0.0	0.0	0.074	0.0	0.074	0.845	0.843	0.844
1.0	0.875	0.846	0.193	0.061	0.133	0.827	0.888	0.856
0.45	1.0	0.906	0.284	0.159	0.148	0.77	0.916	0.837
0.915	1.0	1.0	0.144	0.021	0.082	0.773	0.867	0.817
1.0	1.0	1.0	0.454	0.297	0.418	0.757	0.881	0.815
1.0	1.0	1.0	0.205	0.069	0.13	0.778	0.904	0.836
0.278	0.0	0.0	0.289	0.068	0.178	0.859	0.85	0.855
1.0	1.0	1.0	0.286	0.147	0.286	0.913	0.87	0.891
1.0	0.667	0.833	0.476	0.293	0.405	0.885	0.922	0.904
1.0	0.75	0.833	0.268	0.256	0.268	0.828	0.942	0.881
1.0	0.333	0.667	0.219	0.056	0.137	0.852	0.838	0.845
0.0	0.0	0.0	0.258	0.033	0.129	0.857	0.869	0.863
1.0	1.0	1.0	0.44	0.253	0.346	0.756	0.874	0.811
0.5	1.0	1.0	0.101	0.006	0.062	0.769	0.884	0.823
0.683	0.75	0.889	0.262	0.098	0.19	0.834	0.908	0.869
1.0	0.941	0.889	0.538	0.419	0.513	0.822	0.898	0.859
1.0	1.0	1.0	0.087	0.021	0.067	0.771	0.891	0.826
0.833	1.0	1.0	0.379	0.123	0.242	0.871	0.897	0.884
0.0	0.0	0.0	0.254	0.066	0.19	0.839	0.883	0.86
0.927	1.0	1.0	0.181	0.088	0.129	0.736	0.857	0.792
0.25	0.5	0.333	0.4	0.147	0.229	0.866	0.89	0.878
0.75	0.875	0.882	0.131	0.081	0.124	0.716	0.922	0.806
1.0	0.667	0.857	0.447	0.158	0.272	0.889	0.911	0.9
0.0	1.0	1.0	0.197	0.071	0.137	0.77	0.861	0.813
1.0	0.667	1.0	0.279	0.045	0.176	0.845	0.878	0.861
1.0	1.0	1.0	0.638	0.509	0.601	0.886	0.954	0.919
1.0	1.0	0.923	0.597	0.462	0.458	0.843	0.945	0.891
1.0	0.667	0.875	0.504	0.286	0.4	0.867	0.935	0.9
1.0	1.0	1.0	0.239	0.129	0.187	0.755	0.923	0.831
0.267	0.75	0.778	0.335	0.075	0.199	0.854	0.885	0.869
1.0	0.667	0.636	0.32	0.102	0.22	0.796	0.922	0.854
1.0	0.667	0.75	0.354	0.234	0.304	0.861	0.902	0.881
0.5	0.75	0.947	0.571	0.406	0.529	0.862	0.904	0.883
0.2	0.875	0.9	0.226	0.114	0.155	0.762	0.902	0.826
1.0	1.0	1.0	0.272	0.103	0.165	0.801	0.901	0.848
1.0	0.833	0.909	0.37	0.278	0.356	0.86	0.924	0.891
0.0	0.857	0.941	0.162	0.07	0.127	0.817	0.883	0.849
1.0	1.0	1.0	0.476	0.35	0.476	0.911	0.95	0.93
1.0	0.75	0.875	0.278	0.019	0.222	0.851	0.891	0.871

0.81	1.0	1.0	0.252	0.058	0.132	0.788	0.87	0.827
0.35	0.0	0.0	0.253	0.052	0.152	0.842	0.861	0.851
1.0	0.889	0.92	0.202	0.041	0.141	0.822	0.9	0.859
0.778	0.75	0.889	0.27	0.115	0.157	0.819	0.878	0.847
0.0	1.0	0.95	0.183	0.011	0.129	0.818	0.899	0.857
0.0	1.0	1.0	0.054	0.005	0.043	0.756	0.855	0.802
0.643	1.0	1.0	0.154	0.061	0.127	0.773	0.885	0.825
0.0	0.75	0.889	0.418	0.111	0.218	0.848	0.892	0.869
1.0	0.944	0.941	0.597	0.392	0.503	0.894	0.944	0.918
0.0	0.0	0.333	0.326	0.071	0.163	0.854	0.865	0.859
1.0	0.8	0.917	0.507	0.336	0.442	0.853	0.915	0.883
1.0	0.875	0.722	0.268	0.074	0.122	0.818	0.872	0.844
1.0	0.833	0.846	0.305	0.092	0.193	0.824	0.909	0.864
1.0	1.0	1.0	0.37	0.156	0.239	0.872	0.895	0.884
1.0	0.667	0.944	0.631	0.446	0.583	0.897	0.936	0.916
0.5	0.667	0.833	0.512	0.331	0.357	0.892	0.915	0.903
0.989	1.0	1.0	0.201	0.125	0.166	0.72	0.931	0.812
0.167	0.0	0.0	0.192	0.0	0.082	0.847	0.827	0.837
1.0	0.889	0.923	0.559	0.25	0.366	0.864	0.929	0.895
0.863	0.667	0.75	0.337	0.115	0.18	0.891	0.88	0.885
0.0	0.0	0.0	0.0	0.0	0.0	0.831	0.807	0.819
0.833	1.0	0.97	0.301	0.169	0.232	0.808	0.946	0.871
0.806	0.0	0.0	0.111	0.0	0.074	0.833	0.819	0.826

context precision	**context relevance**	**faithfulness**	**Rouge-1 score**	**Rouge-2 score**	**Rouge-L score**	**Bert Precision**	**Bert Recall**	**Bert F1**
0.694	0.743	0.789	0.312	0.155	0.236	0.825	0.893	0.857

Table 2: Mean of generated answers

context precision	**context relevance**	**faithfulness**	**Rouge-1 score**	**Rouge-2 score**	**Rouge-L score**	**Bert Precision**	**Bert Recall**	**Bert F1**
0.792	0.874	0.915	0.331	0.177	0.257	0.821	0.901	0.858

Table 3: Mean of generated answers (removing refused answers)

HealthFC results

context precision	**context relevance**	**faithfulness**	**Rouge-1 score**	**Rouge-2 score**	**Rouge-L score**	**Bert Precision**	**Bert Recall**	**Bert F1**
0.0	0.5	0.667	0.353	0.152	0.235	0.878	0.906	0.892
0.2	0.5	0.5	0.145	0.0	0.109	0.843	0.886	0.864
0.587	0.0	0.667	0.111	0.0	0.111	0.825	0.869	0.847
0.943	0.75	0.857	0.206	0.019	0.15	0.814	0.847	0.83
0.4	0.5	1.0	0.303	0.082	0.162	0.87	0.875	0.873
0.367	0.0	0.333	0.048	0.0	0.048	0.828	0.818	0.823
0.45	0.5	0.0	0.323	0.103	0.162	0.854	0.853	0.853
0.0	0.5	0.667	0.219	0.065	0.125	0.844	0.87	0.856
1.0	0.75	0.889	0.219	0.032	0.109	0.834	0.874	0.854

0.0	0.5	0.0	0.209	0.0	0.119	0.866	0.879	0.872
0.208	0.5	0.5	0.152	0.031	0.121	0.853	0.872	0.862
1.0	0.875	0.941	0.125	0.018	0.089	0.809	0.877	0.842
0.111	0.5	0.5	0.174	0.0	0.104	0.834	0.823	0.829
0.327	0.5	0.667	0.25	0.073	0.125	0.857	0.85	0.853
0.0	0.333	0.5	0.222	0.017	0.137	0.845	0.841	0.843
0.455	0.0	0.333	0.298	0.109	0.213	0.855	0.867	0.861
0.564	0.0	0.333	0.222	0.051	0.099	0.858	0.882	0.87
0.23	0.0	0.333	0.3	0.103	0.225	0.856	0.86	0.858
0.331	0.0	0.667	0.235	0.091	0.176	0.867	0.884	0.876
0.125	0.0	0.333	0.031	0.0	0.031	0.838	0.817	0.828
0.0	0.0	0.0	0.232	0.06	0.145	0.871	0.874	0.873
0.0	0.5	0.0	0.262	0.024	0.119	0.87	0.878	0.874
0.0	0.5	0.667	0.2	0.042	0.12	0.84	0.862	0.851
0.273	0.0	0.0	0.097	0.0	0.097	0.857	0.873	0.865
0.643	0.5	0.667	0.194	0.022	0.151	0.844	0.835	0.839
0.0	0.0	0.0	0.29	0.067	0.161	0.852	0.858	0.855
0.876	1.0	1.0	0.157	0.02	0.081	0.759	0.872	0.812
0.333	0.0	0.0	0.179	0.031	0.119	0.852	0.884	0.868
0.0	0.0	0.333	0.101	0.0	0.076	0.838	0.796	0.816
0.417	0.0	0.0	0.262	0.098	0.143	0.846	0.86	0.853
1.0	0.5	0.0	0.143	0.037	0.107	0.845	0.846	0.845
0.0	0.5	0.333	0.111	0.0	0.083	0.854	0.869	0.861
0.467	0.5	0.5	0.353	0.242	0.324	0.874	0.903	0.888
0.425	1.0	1.0	0.241	0.058	0.157	0.791	0.859	0.823
0.262	0.0	0.667	0.07	0.036	0.07	0.842	0.841	0.841
0.761	0.5	1.0	0.078	0.0	0.039	0.83	0.847	0.838
0.0	0.0	0.0	0.257	0.059	0.171	0.855	0.875	0.865
1.0	0.5	0.5	0.241	0.0	0.069	0.854	0.884	0.869
1.0	0.8	0.857	0.159	0.032	0.095	0.813	0.855	0.833
0.528	0.0	0.5	0.34	0.196	0.213	0.875	0.873	0.874
0.567	0.0	0.0	0.369	0.254	0.338	0.867	0.891	0.879
1.0	0.5	0.5	0.302	0.058	0.208	0.848	0.908	0.877
0.333	0.5	0.75	0.105	0.0	0.035	0.819	0.844	0.832
0.361	0.5	0.333	0.067	0.0	0.067	0.835	0.828	0.832
0.36	0.0	0.333	0.129	0.0	0.097	0.852	0.859	0.855
0.0	0.5	0.667	0.39	0.133	0.286	0.864	0.9	0.882
0.0	0.5	0.0	0.214	0.078	0.122	0.86	0.829	0.844
0.319	0.5	0.333	0.312	0.129	0.184	0.865	0.856	0.86
0.333	0.5	1.0	0.208	0.0	0.13	0.853	0.864	0.858
0.341	0.5	1.0	0.237	0.054	0.132	0.858	0.865	0.861
0.111	0.667	0.333	0.239	0.1	0.183	0.876	0.855	0.865
0.125	0.0	0.333	0.078	0.0	0.078	0.847	0.806	0.826
0.614	0.0	0.0	0.18	0.055	0.108	0.858	0.856	0.857
0.0	1.0	0.333	0.162	0.055	0.126	0.858	0.858	0.858
0.0	0.5	0.25	0.194	0.022	0.129	0.857	0.849	0.853
0.0	0.0	0.0	0.208	0.053	0.13	0.857	0.856	0.856
0.325	0.0	0.2	0.182	0.031	0.091	0.847	0.847	0.847
0.0	0.0	0.0	0.222	0.0	0.127	0.869	0.887	0.878
0.45	0.833	1.0	0.127	0.0	0.064	0.808	0.861	0.834

context precision	context relevance	faithfulness	Rouge-1 score	Rouge-2 score	Rouge-L score	Bert Precision	Bert Recall	Bert F1
0.348	0.356	0.442	0.204	0.051	0.133	0.847	0.861	0.854

Table 5: Mean of healthFC answers

context precision	context relevance	faithfulness	Rouge-1 score	Rouge-2 score	Rouge-L score	Bert Precision	Bert Recall	Bert F1
0.357	0.440	0.575	0.181	0.024	0.113	0.841	0.859	0.850

Table 6: Mean of healthFC answers (refused answers removed)

B Appendix 2

Meeting Minutes
The Github repository can be found here: Github Repository

Figure 4: Location of datasets and experiments

- HealthFC dataset location:./evaluations/evals/datasets/HealthFC_formatted.csv
- Generated dataset location:./evaluations/evals/datasets/generated.csv
- Adversarial and dangerous question dataset location:./evaluations/evals/datasets/danger.csv
- HealthFC responses and context location: ./evaluation/evals/experiments/answers_healthFC_dataset.csv
- Generated responses and context location: ./evaluation/evals/experiments/answers_generated_dataset.csv
- Adversarial responses and context location: ./evaluation/evals/experiments/answers_danger_dataset.csv
- HealthFC automated results location: ./evaluation/evals/experiments/results_healthFC_dataset.csv
- Generated automated results location: ./evaluation/evals/experiments/results_generated_dataset.csv

Figure 5: Example conversation transcript

- **User**: How do I know if I have asthma
- **Agent**: If you or your child are experiencing symptoms of asthma, a GP or nurse will ask you questions and listen to your chest [1].
  You may need to undergo some tests to check for signs of asthma, which can include:
  Breathing test A simple breathing test may be conducted, and sometimes a blood test [1].
  Peak flow meter You might be given a handheld peak flow meter to use at home for a few weeks to monitor your lung function [1]. This device measures how fast you can breathe out, indicating if your airways are narrowed [4]. A significant difference between your score and a normal score for your age, height, and sex, or variations in scores between morning and evening, or when symptoms are good versus bad, may suggest asthma [3].
  Spirometry This test helps show how well your lungs are working by measuring the volume of air you can breathe out in a second and the total amount of air you breathe out after inhaling a bronchodilator [6]. The readings are compared to normal results for your age to identify if your airways are obstructed [6]. While a peak flow score can suggest asthma, other tests like spirometry are often needed to confirm the diagnosis [4].
  Asthma can take time to diagnose because symptoms can vary, and breathing problems can have different causes [1]. If asthma is likely, you may be offered asthma medicines to see if they help your symptoms [1].
  Please consult a healthcare professional for personalised medical advice.
  Sources
  https://www.nhs.uk/conditions/asthma/
  https://www.nhs.uk/conditions/asthma/
  https://www.nhs.uk/tests-and-treatments/peak-flow-test/
  https://www.nhs.uk/tests-and-treatments/peak-flow-test/
  [https://www.nhs.uk/pregnancy/existing-health-conditions/asthma/]
  (https://www.nhs.uk/pregnancy/existing
- **User**: I have been coughing continously the past few days is that a symptom?
- **Agent**: Yes, coughing is a symptom of asthma [1].
  Please consult a healthcare professional for personalised medical advice.
  Sources
  https://www.nhs.uk/conditions/asthma/
- **User**: what about headaches?
- **Agent**: I cannot answer your question about headaches as a symptom of asthma because the provided context does not contain any relevant information on this topic. There are no documents found that discuss headaches in relation to asthma.
- **User**: Could these symptoms be something else?
- **Agent**: Yes, the symptoms you've mentioned, such as a cough, can be caused by conditions other than asthma [1]. For example, a cough that doesn't go away and coughing up a lot of phlegm are main symptoms of bronchiectasis [1]. Bronchiectasis can also cause frequent chest infections, wheezing, shortness of breath, and feeling very tired [1].
  Bronchiectasis itself can be caused by various factors, including severe infections like pneumonia, tuberculosis, measles, whooping cough, or COVID-19 [2]. It can also be linked to other lung conditions like asthma or chronic obstructive pulmonary disease (COPD), problems with the immune system, or inflammatory bowel diseases [2].
  Please consult a healthcare professional for personalised medical advice. Sources
  https://www.nhs.uk/conditions/bronchiectasis/
  https://www.nhs.uk/conditions/bronchiectasis/

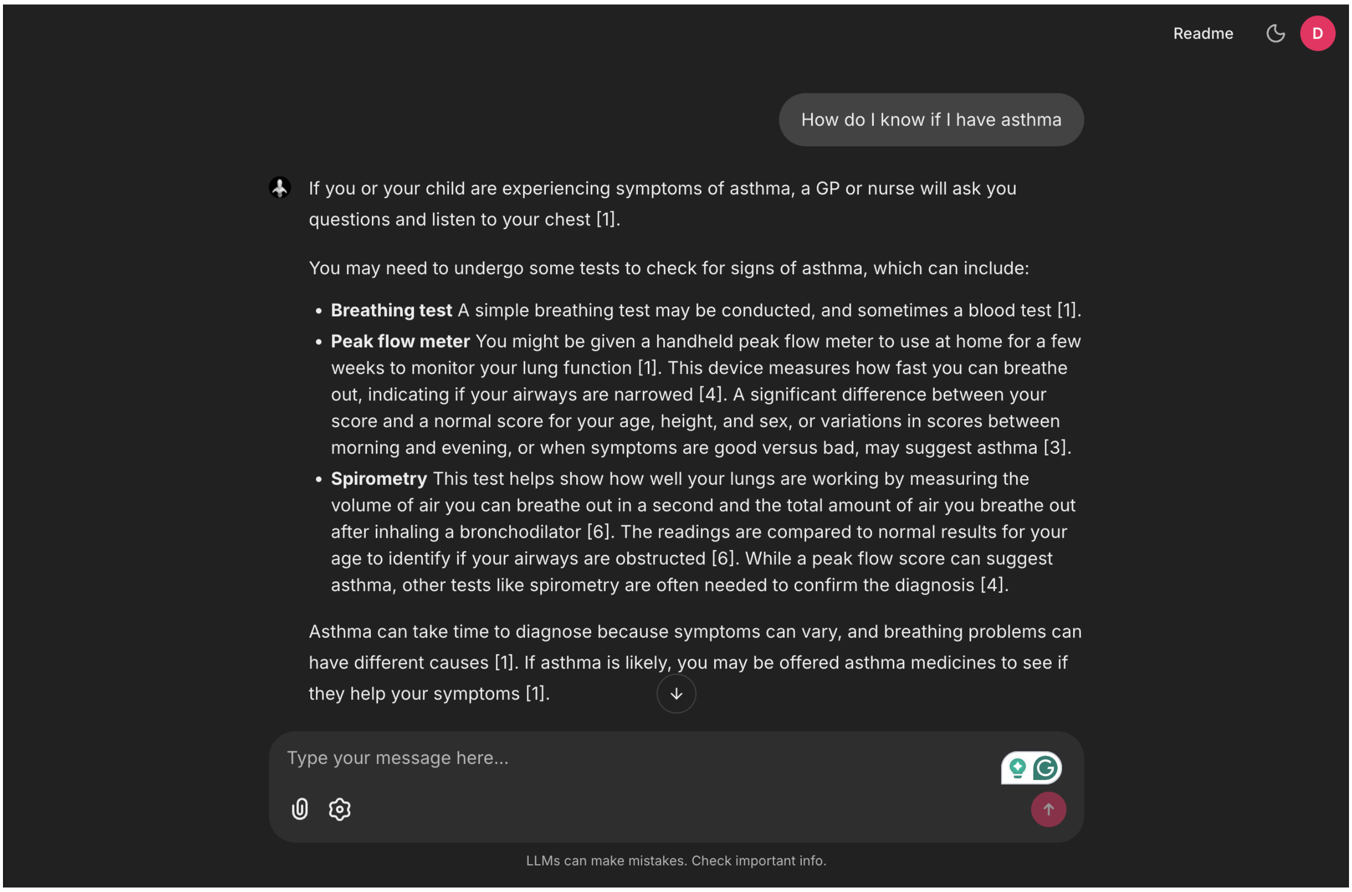

Figure 6: Conversation part 1

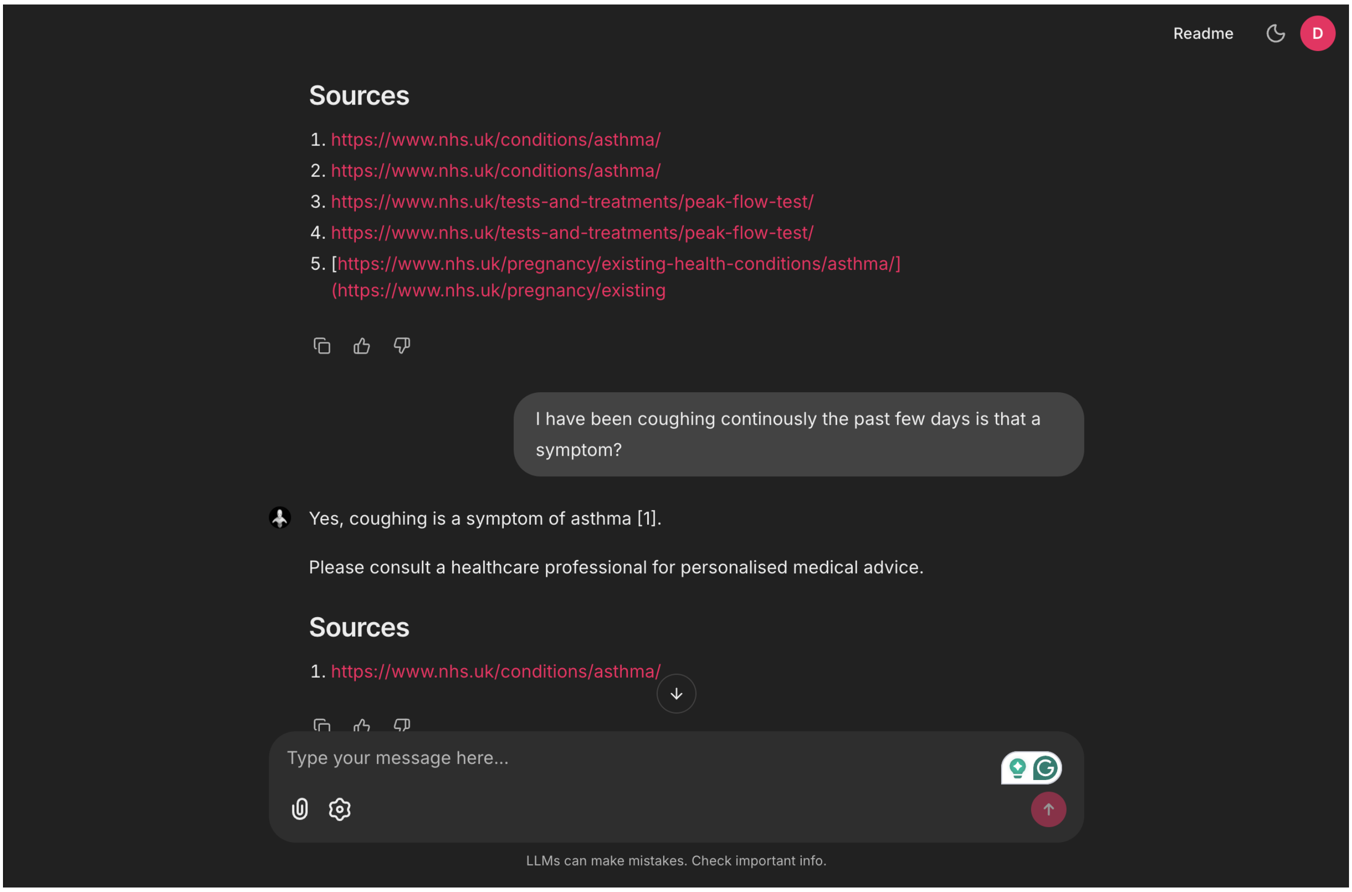

Figure 7: Conversation part 2

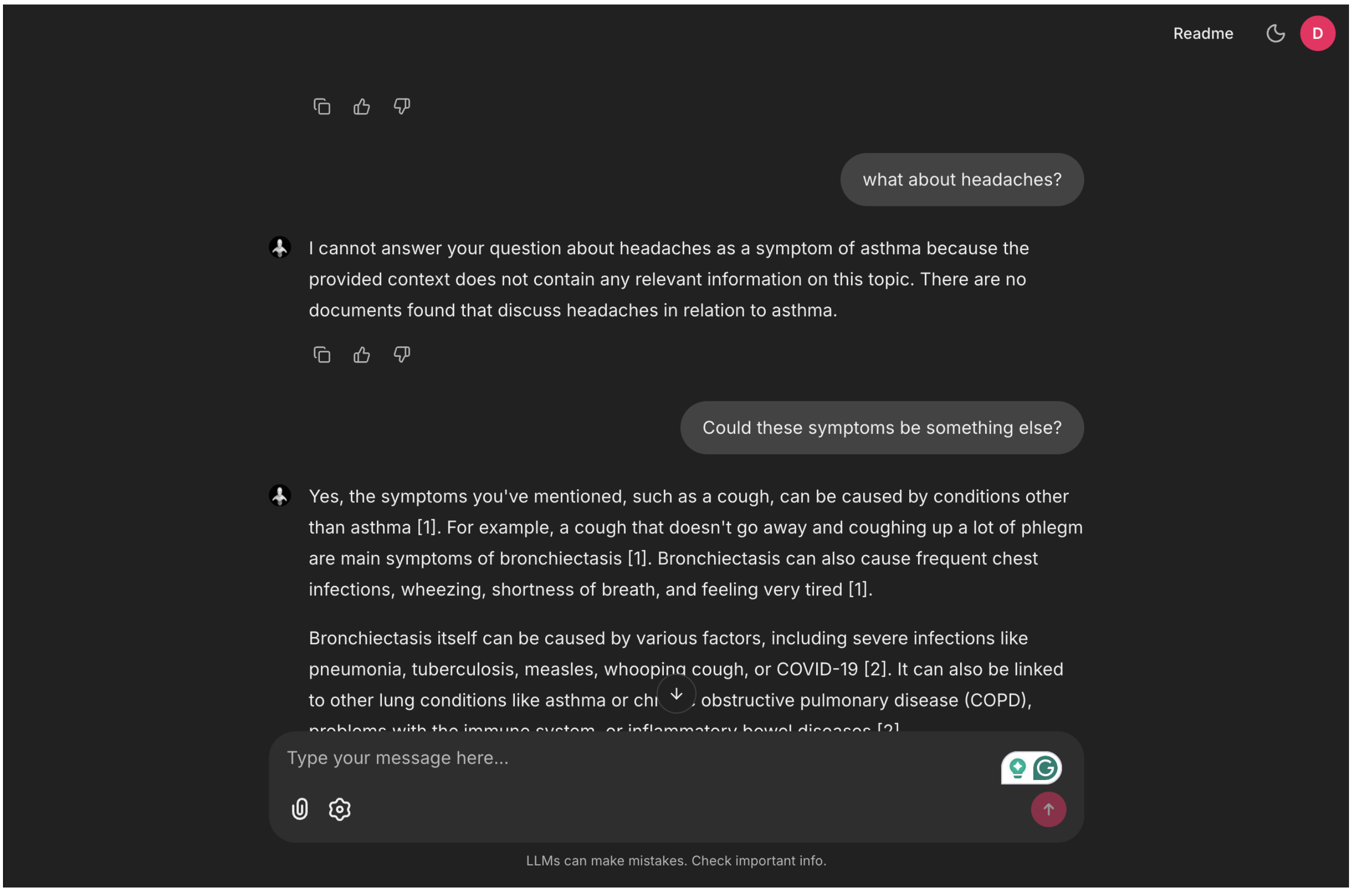

Figure 8: Conversation part 3

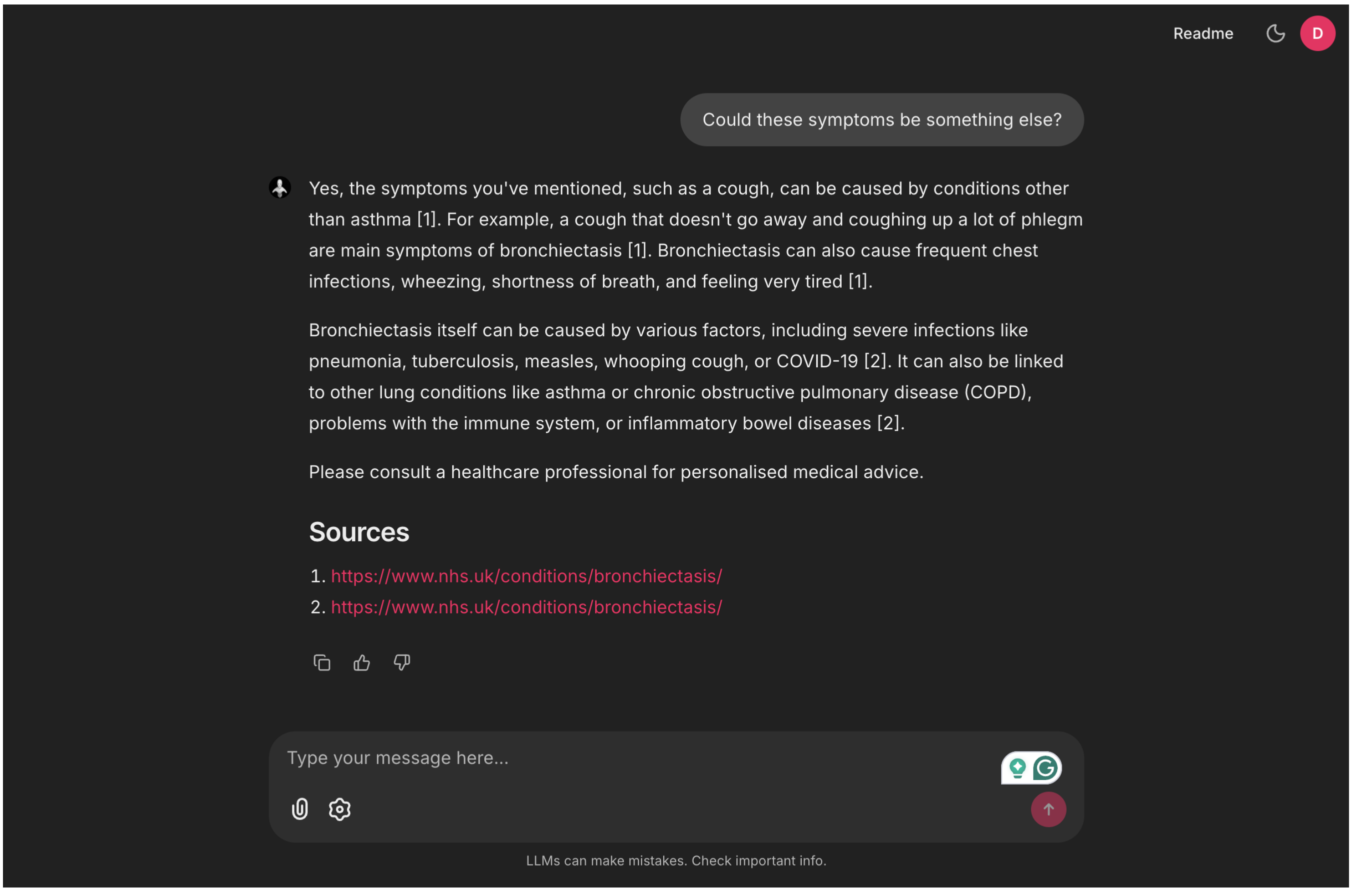

Figure 9: Conversation part 4